

Lab Experiment: 03

1. Singly Linked List Implementation

```
#include <stdio.h>

#include <stdlib.h>

// Node structure for the singly linked list
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Insertion at the beginning
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}
```

// Insertion at the end

```
void insertAtEnd(struct Node** head, int data) {  
    struct Node* newNode = createNode(data);  
    if (*head == NULL) {  
        *head = newNode;  
        return;  
    }  
    struct Node* temp = *head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}
```

// Insertion at specific position

```
void insertAtPosition(struct Node** head, int data, int position) {  
    if (position < 1) {  
        printf("Invalid position!\n");  
        return;  
    }  
    if (position == 1) {  
        insertAtBeginning(head, data);  
        return;  
    }  
    struct Node* newNode = createNode(data);  
    struct Node* temp = *head;  
    for (int i = 1; i < position - 1 && temp != NULL; i++) {  
        temp = temp->next;  
    }  
}
```

```
if (temp == NULL) {  
    printf("Position out of bounds\n");  
    free(newNode);  
    return;  
}  
newNode->next = temp->next;  
temp->next = newNode;  
}
```

// Deletion at beginning

```
void deleteFromBeginning(struct Node** head) {  
    if (*head == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    struct Node* temp = *head;  
    *head = (*head)->next;  
    free(temp);  
}
```

// Deletion at end

```
void deleteFromEnd(struct Node** head) {  
    if (*head == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    struct Node* temp = *head;  
    if (temp->next == NULL) {  
        free(*head);  
    }
```

```

    *head = NULL;

    return;
}
while (temp->next->next != NULL) {
    temp = temp->next;
}
free(temp->next);
temp->next = NULL;
}

```

//Deletion at specific position

```

void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL || position < 1) {
        printf("List is empty or invalid position\n");
        return;
    }
    struct Node* temp = *head;
    if (position == 1) {
        deleteFromBeginning(head);
        return;
    }
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of bounds\n");
        return;
    }
    struct Node* nodeToDelete = temp->next;

```

```
temp->next = temp->next->next;
free(nodeToDelete);
}
```

// Function to display the linked list

```
void displayList(struct Node* head) {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```
int main() {
    struct Node* head = NULL;
    int choice, data, position;

    do {
        printf("\n--- Singly Linked List Menu ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Position\n");
        printf("4. Delete from Beginning\n");
```

```
printf("5. Delete from End\n");
printf("6. Delete from Position\n");
printf("7. Display List\n");
printf("8. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        printf("Enter data: ");
        scanf("%d", &data);
        insertAtBeginning(&head, data);
        break;
    case 2:
        printf("Enter data: ");
        scanf("%d", &data);
        insertAtEnd(&head, data);
        break;
    case 3:
        printf("Enter data: ");
        scanf("%d", &data);
        printf("Enter position: ");
        scanf("%d", &position);
        insertAtPosition(&head, data, position);
        break;
    case 4:
        deleteFromBeginning(&head);
        break;
    case 5:
```

```
        deleteFromEnd(&head);

        break;
case 6:
    printf("Enter position: ");
    scanf("%d", &position);
    deleteFromPosition(&head, position);

    break;
case 7:
    displayList(head);

    break;
case 8:
    printf("Exiting...\n");

    break;
default:
    printf("Invalid choice. Please try again.\n");

}
} while (choice != 8);

return 0;
}
```

```

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List
8. Exit
Enter your choice: 1
Enter data: 12

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List
8. Exit
Enter your choice: 3
Enter data: 33
Enter position: 2

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List
8. Exit

Enter your choice: 2
Enter data: 11

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List
8. Exit
Enter your choice: 6
Enter position: 2

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List
8. Exit
Enter your choice: 7
Linked List: 12 -> 11 -> NULL

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List
8. Exit
Enter your choice: 8
Exiting...

```

2. Doubly Linked List Implementation:

```

#include <stdio.h>
#include <stdlib.h>

// Node structure for the doubly linked list
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;

```



```

newNode->next = NULL;
newNode->prev = NULL;
return newNode;
}

```

```

// Function to insert at the beginning
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head != NULL) {
        (*head)->prev = newNode;
    }
    newNode->next = *head;
    *head = newNode;
}

```

```

// Function to insert at the end
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}

```

```

// Function to insert at a specific position (1-based index)
void insertAtPosition(struct Node** head, int data, int position) {
    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }
    if (position == 1) {
        insertAtBeginning(head, data);
        return;
    }
    struct Node* newNode = createNode(data);
    struct Node* temp = *head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
}

```

```

    }
    if (temp == NULL) {
        printf("Position out of bounds\n");
        free(newNode);
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
}

```

```

// Function to delete from the beginning
void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *head;
    *head = (*head)->next;
    if (*head != NULL) {
        (*head)->prev = NULL;
    }
    free(temp);
}

```

```

// Function to delete from the end
void deleteFromEnd(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *head;
    if (temp->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->prev->next = NULL;
}

```

```

    free(temp);
}

// Function to delete from a specific position (1-based index)
void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL || position < 1) {
        printf("List is empty or invalid position\n");
        return;
    }
    struct Node* temp = *head;
    if (position == 1) {
        deleteFromBeginning(head);
        return;
    }
    for (int i = 1; i < position && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position out of bounds\n");
        return;
    }
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }
    free(temp);
}

// Function to display the linked list forward
void displayListForward(struct Node* head) {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Linked List (Forward): ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

// Function to display the linked list backward
void displayListBackward(struct Node* head) {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    printf("Linked List (Backward): ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}

```

```

int main() {
    struct Node* head = NULL;
    int choice, data, position;

    do {
        //printf("\n--- Doubly Linked List Menu ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Position\n");
        printf("4. Delete from Beginning\n");
        printf("5. Delete from End\n");
        printf("6. Delete from Position\n");
        printf("7. Display List Forward\n");
        printf("8. Display List Backward\n");
        printf("9. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                insertAtBeginning(&head, data);
                break;
            case 2:

```

```

        printf("Enter data: ");
        scanf("%d", &data);
        insertAtEnd(&head, data);
        break;
    case 3:
        printf("Enter data: ");
        scanf("%d", &data);
        printf("Enter position: ");
        scanf("%d", &position);
        insertAtPosition(&head, data, position);
        break;
    case 4:
        deleteFromBeginning(&head);
        break;
    case 5:
        deleteFromEnd(&head);
        break;
    case 6:
        printf("Enter position: ");
        scanf("%d", &position);
        deleteFromPosition(&head, position);
        break;
    case 7:
        displayListForward(head);
        break;
    case 8:
        displayListBackward(head);
        break;
    case 9:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice. Please try again.\n");
    }
} while (choice != 9);

return 0;
}

```

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 1

Enter data: 12

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 2

Enter data: 33

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 3

Enter data: 4

Enter position: 1

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 2

Enter data: 51

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 6

Enter position: 2

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 7

Linked List (Forward): 4 -> 33 -> 51 -> NULL

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 8

Linked List (Backward): 51 -> 33 -> 4 -> NULL

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display List Forward
8. Display List Backward
9. Exit
```

Enter your choice: 9

Exiting...

3. Application Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// Define the structure for a task node in the to-do list
struct Task {
    char description[100];
    struct Task* next;
};
// Head pointer to point to the first task in the to-do list
struct Task* head = NULL;
// Function to add a new task at the end of the to-do list
void addTask(char* description) {
    struct Task* newTask = (struct Task*)malloc(sizeof(struct Task));
    strcpy(newTask->description, description);
    newTask->next = NULL;
    if (head == NULL) {
        head = newTask;
    } else {
        struct Task* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newTask;
    }
    printf("Task added: %s\n", description);
}
// Function to mark a task as completed by removing it from the to-do list
void completeTask(int position) {
    if (head == NULL) {
        printf("To-do list is empty\n");
        return;
    }
    struct Task* temp = head;
    // If the task to be completed is the first task
    if (position == 1) {
        head = head->next;
        printf("Completed task: %s\n", temp->description);
        free(temp);
        return;
    }
    // Traverse to the specified position
    struct Task* prev = NULL;
    for (int i = 1; i < position && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
```

```

printf("Invalid position! Task not found.\n");
} else {
prev->next = temp->next;
printf("Completed task: %s\n", temp->description);
free(temp);
}
}
// Function to display all tasks in the to-do list
void displayTasks() {
struct Task* temp = head;
if (temp == NULL) {
printf("To-do list is empty\n");
return;
}
int taskNumber = 1;
printf("To-Do List:\n");
while (temp != NULL) {
printf("%d. %s\n", taskNumber, temp->description);
temp = temp->next;
taskNumber++;
}
}
// Main function with a menu-driven interface
int main() {
int choice, position;
char description[100];
while (1) {
printf("\nTo-Do List Menu:\n");
printf("1. Add Task\n");
printf("2. Complete Task\n");
printf("3. Display Tasks\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
getchar(); // To consume the newline character left by scanf
switch (choice) {
case 1:
printf("Enter task description: ");
fgets(description, sizeof(description), stdin);
description[strcspn(description, "\n")] = '\0'; // Remove trailing newline
addTask(description);
break;
case 2:
printf("Enter task number to complete: ");
scanf("%d", &position);
completeTask(position);
break;
case 3:
displayTasks();

```



```

break;
case 4:
exit(0);
default:
printf("Invalid choice. Please try again.\n");
}
}
return 0;
}

```

```

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 1
Enter task description: make parents proud
Task added: make parents proud

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 1
Enter task description: complete experiments
Task added: complete experiments

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 3
To-Do List:
1. make parents proud
2. complete experiments

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 2
Enter task number to complete: 1
Completed task: make parents proud

```

4. Memory Usage and Dynamic Allocation:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// Define the structure for a task node in the to-do list
struct Task {

```

```

char description[100];
struct Task* next;
};
// Head pointer to point to the first task in the to-do list
struct Task* head = NULL;
// Function to add a new task at the end of the to-do list
void addTask(char* description) {
    struct Task* newTask = (struct Task*)malloc(sizeof(struct Task));
    if (newTask == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    strcpy(newTask->description, description);
    newTask->next = NULL;
    if (head == NULL) {
        head = newTask;
    } else {
        struct Task* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newTask;
    }
    printf("Task added: %s\n", description);
}
// Function to mark a task as completed by removing it from the to-do list
void completeTask(int position) {
    if (head == NULL) {
        printf("To-do list is empty\n");
        return;
    }
    struct Task* temp = head;
    // If the task to be completed is the first task
    if (position == 1) {
        head = head->next;
        printf("Completed task: %s\n", temp->description);
        free(temp); // Free the memory of the completed task
        return;
    }
    // Traverse to the specified position
    struct Task* prev = NULL;
    for (int i = 1; i < position && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
    }
}

```

```

}
if (temp == NULL) {
printf("Invalid position! Task not found.\n");
} else {
prev->next = temp->next;
printf("Completed task: %s\n", temp->description);
free(temp); // Free the memory of the completed task
}
}

// Function to display all tasks in the to-do list
void displayTasks() {
struct Task* temp = head;
if (temp == NULL) {
printf("To-do list is empty\n");
return;
}
int taskNumber = 1;
printf("To-Do List:\n");
while (temp != NULL) {
printf("%d. %s\n", taskNumber, temp->description);
temp = temp->next;
taskNumber++;
}
}

// Function to free all tasks in the to-do list (used before program exits)
void freeAllTasks() {
struct Task* temp = head;
while (temp != NULL) {
struct Task* nextTask = temp->next;
printf("Freeing task: %s\n", temp->description);
free(temp); // Free each task's memory
temp = nextTask;
}
head = NULL; // Set head to NULL after all tasks are freed
}

// Main function with a menu-driven interface
int main() {
int choice, position;
char description[100];
while (1) {
printf("\nTo-Do List Menu:\n");
printf("1. Add Task\n");
printf("2. Complete Task\n");
printf("3. Display Tasks\n");

```

```
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
getchar(); // To consume the newline character left by scanf
switch (choice) {
case 1:
printf("Enter task description: ");
fgets(description, sizeof(description), stdin);
description[strcspn(description, "\n")] = '\0'; // Remove trailing newline
addTask(description);
break;
case 2:
printf("Enter task number to complete: ");
scanf("%d", &position);
completeTask(position);
break;
case 3:
displayTasks();
break;
case 4:
freeAllTasks(); // Free all allocated memory before exiting
exit(0);
default:
printf("Invalid choice. Please try again.\n");
}
}
return 0;
}
```

```
To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice:
1
Enter task description: Complete assignment
Task added: Complete assignment

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 1
Enter task description: Eat healthy
Task added: Eat healthy

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 1
Enter task description: Wash clothes
Task added: Wash clothes

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 2
Enter task number to complete: 3
Completed task: Wash clothes
```

```
To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 3
To-Do List:
1. Complete assignment
2. Eat healthy

To-Do List Menu:
1. Add Task
2. Complete Task
3. Display Tasks
4. Exit
Enter your choice: 4
Freeing task: Complete assignment
Freeing task: Eat healthy

=== Code Execution Successful ===
```